**Algorithm Analysis Fundamentals**

Time complexity analysis of iterative and recursive algorithms; verify Big-O, Ω, Θ for sample programs

Dr. M.Manikandan

## Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** `[10, 25, 30, 45, 50]`, Key = `30`
- **Sample Output:** `Key found at index 2`
- **Complexity:** Ω: O(1), O: O(n), Θ: Θ(n)
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

## Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** `[5, 10, 15, 20, 25]`, Key = `20`
- **Sample Output:** `Key found at index 3`
- **Complexity:** Ω: O(1), O: O(logn), Θ: Θ(logn)
- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

## Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** `n = 5`
- **Sample Output:** `120`
- **Complexity:** Both O(n)
- **Explanation:** Highlights recursion depth vs iterative loops.

## Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** `n = 6`
- **Sample Output:** `[0, 1, 1, 2, 3, 5]`
- **Complexity:** Iterative O(n), Recursive naïve O(2n), Recursive with memoization O(n)
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

## Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** `A=[[1,2],[3,4]], B=[[5,6],[7,8]]`
- **Sample Output:** `[[19,22],[43,50]]`
- **Complexity:** O(n3)
- **Explanation:** Shows cubic growth due to nested loops.

## Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** `[38, 27, 43, 3, 9, 82, 10]`
- **Sample Output:** `[3, 9, 10, 27, 38, 43, 82]`
- **Complexity:** Always O(nlogn)
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.

## Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** `[10, 7, 8, 9, 1, 5]`
- **Sample Output:** `[1, 5, 7, 8, 9, 10]`
- **Complexity:** Ω: O(nlogn), O: O(n2), Θ: Θ(nlogn)
- **Explanation:** Shows how pivot choice affects performance.

## Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** `n = 3`
- **Sample Output:** Sequence of moves.
- **Complexity:** O(2n)
- **Explanation:** Illustrates exponential growth in recursive calls.

## Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** `a=48, b=18`
- **Sample Output:** 6
- **Complexity:** O(logn)
- **Explanation:** Shows efficient recursion with logarithmic complexity.

## Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** `[12, 11, 13, 5, 6]`
- **Sample Output:** `[5, 6, 11, 12, 13]`
- **Complexity:** Ω: O(n), O: O(n2), Θ: Θ(n2)
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

## Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** `[4, 10, 3, 5, 1]`
- **Sample Output:** `[1, 3, 4, 5, 10]`
- **Complexity:** Always O(nlogn)
- **Explanation:** Consistent performance using heap structure.

## Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** `[2, 4, 1, 3, 5]`
- **Sample Output:** 3
- **Complexity:** O(nlogn)
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

## Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** `[-2, -3, 4, -1, -2, 1, 5, -3]`
- **Sample Output:** 7
- **Complexity:** Kadane's O(n), Divide & Conquer O(nlogn)
- **Explanation:** Highlights efficiency differences between approaches.

## Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute xn.
- **Sample Input:** `x=2, n=10`
- **Sample Output:** `1024`
- **Complexity:** Iterative O(n), Recursive fast power O(logn)
- **Explanation:** Shows how recursion reduces complexity.

## Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** `[(1,2), (4,5), (7,8), (3,1)]`
- **Sample Output:** Closest pair `(1,2)-(3,1)`, Distance = √5

- **Complexity:** Brute force O(n2), Optimized O(nlogn)
- **Explanation:** Demonstrates quadratic complexity and motivates optimization.